ASE CHAPTER 3

T3 Comparison & Analytical Breakdown Pack

Section 1: Core Textbook Comparative & Analytical Exercises (11 Items)

Exercise 3.2. Communication Breakdown in Requirements Analysis

Requirements analysis is unquestionably the most communication intensive step in the software engineering process. Why does the communication path frequently break down?

Analysis of Communication Path Failures: Requirements analysis bridges two fundamentally distinct cognitive domains: domain stakeholders who understand business operations, and software engineers who understand technical architectures. Communication frequently breaks down due to five structural causes:

1. **The Semantic Vocabulary Gap:** Domain users express needs using specialized institutional terminology and acronyms that developers misinterpret, while developers explain constraints using technical abstractions (schemas, threads, protocols) that confuse users.
2. **Tacit and Unstated Knowledge:** Users operate on intuitive assumptions and routine workflows that appear so self-evident to them that they fail to mention them, leaving vital operational rules unstated.
3. **The "Yes But" Phenomenon:** Users rarely know what they truly need until they see an operational system; requirements evolve dynamically only after users begin interacting with software artifacts.
4. **Organizational and Political Conflicts:** Different stakeholder groups within the client organization often hold conflicting objectives (e.g., finance demands strict cost controls, while branch staff demand rapid flexible overrides), forcing the analyst into an unresolved mediation role.
5. **Personnel Turnover and Instability:** Client representatives frequently rotate or leave during long elicitation cycles, resulting in inconsistent instructions and contradictory revisions.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, pp. 40–42; companion Slide 134.

ملاحظة أكاديمية هامة (Academic Note)

التشخيص الهندسي: الفشل في تحليل المتطلبات ليس عيباً تقنياً، بل هو عائق اتصالي بين لغة العمل المؤسسية (*Domain Jargon*) ولغة النظم البرمجية (*Technical Abstraction*)؛ والحل يكمن في النمذجة المرئية المشتركة وورش العمل المنتظمة.

Exercise 3.9. FAST vs. Brainstorming Sessions in Requirements Elicitation

Describe Facilitated Application Specification Technique (FAST) and compare this with brainstorming sessions.

Comparative Breakdown: Both techniques are collaborative elicitation methods designed to overcome communication barriers, but they differ fundamentally in structure, governance, and output:

Dimension	Facilitated Application Specification Technique (FAST)	Brainstorming Sessions
Primary Objective	Build structured, consensual preliminary requirements and draft an initial SRS outline.	Generate a wide, unconstrained volume of creative ideas without immediate filtering.
Structural Formality	Highly structured and formal; strictly adheres to predefined agenda, preparation lists, and facilitation protocols.	Informal and dynamic; operates under open-floor, non-evaluative idea generation rules.
Facilitation Role	Managed by an objective, trained neutral facilitator who enforces rules and drives consensus.	Moderated by a leader whose primary task is simply recording ideas and preventing criticism.
Participant Preparation	Participants must prepare and bring written lists of system objects, services, and constraints.	Participants require minimal upfront preparation; ideas are sparked spontaneously.
Evaluation Timing	Ideas and objects are immediately evaluated, grouped, prioritized, and trimmed during the workshop.	Evaluation is strictly prohibited during ideation; evaluation occurs in a separate follow-up phase.
Deliverable / Output	Prioritized lists of functional requirements, system constraints, and a signed preliminary scope document.	A raw, unranked idea repository requiring subsequent analysis and filtering.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.2–§3.4.3, pp. 51–53; companion Slides 20–26 & Slide 135.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

الفارق الجوهري في الامتحان: العصف الذهني (*Brainstorming*) يمنع النقد تماماً لجمع أكبر قدر من الأفكار الحرة، بينما ورش *FAST* تتطلب تحضيراً مسبقاً وتدمج التقييم والترتيب الفوري للوصول إلى وثيقة مواصفات مشتركة.

Exercise 3.15. Modeling Many-to-Many (M:N) Relationships as Associative Entities

Explain why a many-to-many relationship is to be modeled as an associative entity.

Architectural & Relational Rationale: An associative entity represents the association itself as an entity, which is useful when the relationship has its own attributes or needs to be represented in a relational database.

For example, ENROLLMENT can represent the association between STUDENT and COURSE and hold attributes such as grade or semester. In a relational implementation, it is commonly represented by a

linking table, replacing one M:N relationship with two 1:N relationships. The table's key design depends on the system's rules; a composite key is one possible choice.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, pp. 66–68; companion Slides 87–91 & Slide 136.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **الفكرة:** إذا كانت للعلاقة سمات تخصّها، مثل الدرجة أو الفصل الدراسي في التسجيل، نمثل العلاقة بكيان رابط. وعند تنفيذها كجداول تتحول عادةً إلى علاقتين من نوع N:1.

Exercise 3.16. Linkages Between Data Flow Diagrams (DFD) and E-R Diagrams (ERD)

What are the linkages between data flow and E-R diagrams?

Analysis of Process-Data Duality: DFDs and ERDs show complementary views of a system: a DFD models data moving through processes, while an ERD models data entities and their relationships. They should be checked for consistency rather than treated as having a universal one-to-one mapping.

Data stores and flows in the DFD can be compared with entities, attributes, and data definitions in the ERD and data dictionary. Processes describe operations on data, but a particular store or entity need not map one-to-one to a single object in the other model.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1–§3.5.3, pp. 60–68; companion Slides 82–91 & Slide 136.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **التكامل المعماري:**
>
> - الـDFD يمثل **الأفعال والحركة** (Process Flow).
> - الـERD يمثل **الأسماء والبيانات المستقرة** (Data Persistence).
> - افحص توافق أسماء البيانات في مخازن DFD مع الكيانات والسمات المعرّفة في ERD والقاموس، من دون افتراض تطابق واحد لواحد.

Exercise 3.18. Minimum Cardinality vs. Optional and Mandatory Participation

Explain the relationship between minimum cardinality and optional and mandatory participation.

Comparative Semantic Breakdown: In data modeling, cardinality defines the numeric bounds of an association. It is expressed as a pair: $(\text{minimum cardinality}, \text{maximum cardinality})$.

- **Maximum Cardinality:** Establishes the upper limit of associated entity instances (either 1 or N).
- **Minimum Cardinality:** Establishes whether participation in the relationship is **mandatory** or **optional**.

The Exact Structural Relationship:

1. **Optional Participation ($\text{Min Cardinality} = 0$):**

- An instance of Entity A can exist in the database without being associated with any instance of Entity B.
- *Example:* EMPLOYEE assigned to PROJECT. If a newly hired employee has not yet been assigned a project, the minimum cardinality is 0. Participation of EMPLOYEE is **optional**.

2. **Mandatory Participation ($\text{Min Cardinality} \geq 1$):**
   - Every instance of Entity A must be associated with at least one instance of Entity B; existence without an associated entity violates database integrity constraints.
   - *Example:* BOOK published by PUBLISHER. A book cannot exist in the catalog without an assigned publisher. The minimum cardinality is 1. Participation of BOOK is **mandatory**.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, pp. 66–68; companion Slides 87–91 & Slide 136.

ملاحظة أكاديمية هامة (Academic Note)

قاعدة الحفظ السريعة:

- الحد الأدنى = 0 ← المشاركة اختيارية (*Optional*)؛ السجل يمكن أن يوجد وحيداً.
- الحد الأدنى ≥ 1 ← المشاركة إجبارية (*Mandatory*)؛ لا يمكن إدخال السجل دون ربطه بالكيان الآخر.

Exercise 3.26. Prototyping for Reusable Components: Benefits & Pitfalls

Discuss the significance of using prototyping for reusable components and explain the problems, which may arise in this situation.

Significance of Prototyping for Reusable Components: A prototype can help assess whether a candidate reusable component is suitable for the intended use and can clarify requirements before the final system is built.

Potential problems: Prototyping may consume more effort than expected, encourage unrealistic expectations about delivery, or lead users and developers to misunderstand the prototype's role. Tool and team experience can also affect the usefulness of the results. A prototype should therefore be planned for a clear purpose.

Problems and Pitfalls That Arise:

1. **Premature Architectural Lock-In:** Teams often develop rapid prototypes around a specific component, becoming cognitively biased toward that technology even when the prototype reveals significant architectural flaws.
2. **Superficial Feature Assumption:** A component prototype often tests basic functional workflows successfully while masking critical non-functional deficiencies (e.g., poor scalability, lack of thread safety, missing encryption).
3. **Interface Bloat and Complexity:** Integrating general-purpose reusable libraries often introduces unwanted dependencies, massive memory overheads, and bloated attack surfaces into the application.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.4, pp. 68–71; companion Slide 116 & Slide 138.

ملاحظة أكاديمية هامة (Academic Note)

تنبيه: نجاح prototype لغرض توضيح المتطلبات لا يثبت وحده أن المكوّن أو المنتج النهائي مناسب لكل ظروف التشغيل.

Exercise 3.27. Customer Request to Buy Prototype for Production: Developer's Response

Suppose a user is satisfied with the performance of a prototype. If he/she is interested to buy this for actual work, what should be the response of a developer?

Professional Engineering Response: First identify the prototype's purpose. If it is a throw-away prototype, explain that it was built quickly to clarify requirements and is meant to be discarded; the final system is built separately. If it is an evolutionary or validation prototype intended to become the system, assess whether it is sufficiently complete and efficient for that role before agreeing to use it operationally. A user liking its performance alone does not establish that it is ready for production.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.4, pp. 68–71; companion Slide 116 & Slide 138.

ملاحظة أكاديمية هامة (Academic Note)

الفكرة: رضا المستخدم عن الأداء وحده لا يكفي؛ نوع النموذج وغرضه وجودته هي التي تحدد هل يمكن أن يصبح النظام الفعلي.

Exercise 3.28. Critique: Is "Throw-Away Prototype" an Inappropriate Term?

Comment on the statement: "The term throw-away prototype is inappropriate in that these prototypes expand and enhance the knowledge base that is retained and incorporated in the final prototype; therefore they are not disposed of or thrown away at all."

The statement is partly right: the knowledge gained from a prototype is retained, but that does not mean the prototype artifact itself is retained. In a throw-away approach, the prototype is discarded after analysis and the final system is built separately. In an evolutionary approach, the prototype is intended to evolve into the final product. The term is appropriate when it describes the artifact in the first approach, not the knowledge learned from it.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.4, pp. 68–71; companion Slide 116 & Slide 138.

ملاحظة أكاديمية هامة (Academic Note)

التمييز: ما يُحتفظ به هو المعرفة والمتطلبات؛ أما التخلص من الشيفرة فيخصّ throw-away prototype تحديداً.

Exercise 3.29. Ambiguity Analysis of Four Requirement Statements

Which of the following statements are ambiguous? Explain why. (a) The system shall exhibit good response time. (b) The system shall be menu driven. (c) There shall exist twenty-five buttons on the control panel, numbered PF1 to PF25. (d) The software size shall not exceed 128K of RAM.

Detailed Analytical Evaluation:

1. **Statement (a): "The system shall exhibit good response time."**
   - **Status: Extremely Ambiguous.**
   - **Explanation:** The adjective **"good"** is completely subjective, qualitative, and unmeasurable. It does not define what constitutes good performance (e.g., 50 milliseconds vs. 5 seconds), under what operational workload (single user vs. 10,000 concurrent transactions), or what transaction types are being measured. It is completely untestable and unverifiable.
   - **Unambiguous Revision:** "*The system shall respond to 95% of standard database search queries in less than 1.5 seconds under a concurrent load of 500 active users.*"
2. **Statement (b): "The system shall be menu driven."**
   - **Status: Ambiguous.**
   - **Explanation:** While "menu driven" conveys a general architectural style, it lacks operational specificity. It fails to specify menu hierarchies, navigation mechanics, dynamic vs. static menus, keyboard accessibility shortcuts, or GUI widget specifications.
   - **Example revision (assumed targets):** "*The system shall display the main menu within 2 seconds after login and shall allow users to open each menu item using either the keyboard or mouse.*"
3. **Statement (c): "There shall exist twenty-five buttons on the control panel, numbered PF1 to PF25."**
   - **Status: Unambiguous.**
   - **Explanation:** This statement is mathematically precise, concrete, and directly verifiable. An inspector can physically count the buttons (exactly 25) and verify their labeling (PF1 through PF25). There is no room for subjective interpretation.
4. **Statement (d): "The software size shall not exceed 128K of RAM."**
   - **Status: Unambiguous.**
   - **Explanation:** The constraint is quantitatively bounded and measurable using standard system profiling tools. A tester can load the software and verify whether its memory footprint remains $\leq 128\ \text{KB}$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 73–74; companion Slide 104 & Slide 138.

ملاحظة أكاديمية هامة (Academic Note)

معيار التحقق (Verifiability Rule): إذا كان بإمكانك كتابة فحص محدد بنعم/لا أو رقم كمي لقياس الشرط، فالعبارة سليمة (غير مبهمة). أما الكلمات المطاطية مثل: "جيد"، "سريع"، "سهل الاستخدام"، "حديث" فهي عبارات مبهمة ومرفوضة قطعياً في الـSRS.

Exercise 3.36 (a). Functional vs. Non-Functional Requirements

Discuss the difference between Functional and Non-Functional requirements.

Comparative Dimensional Matrix:

Dimension	Functional Requirements	Non-Functional Requirements
Core Question	*What does the system do?*	*How well does the system perform its functions?*
Definition	Statements of specific services, calculations, inputs, data processing steps, and outputs.	Constraints placed on system services and operational boundaries (performance, quality, security).
Scope of Impact	Typically localized to specific modules, transactions, or use cases.	Global and architectural; influences overall system structure, hardware selection, and data storage.
Verification Method	Verified through functional testing, black-box test cases, and use-case execution.	Verified through specialized stress testing, benchmarking, load testing, security audits, and profiling.
Failure Consequence	Specific service or transaction fails to complete properly.	Entire system may become unusable, insecure, unscalable, or non-compliant, causing total system failure.
Examples	Calculating loan interest; printing grade sheets; processing credit card transactions.	Response time ≤ 2s; 99.99% system availability; AES-256 data encryption at rest.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1–§3.2.2, pp. 45–47; companion Slides 6–8 & Slide 140.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

التأثير المعماري: خطأ المتطلب الوظيفي يعطل ميزة واحدة، لكن إهمال المتطلب غير الوظيفي (مثل الأمان أو التزامن) قد يفرض إعادة كتابة هيكل النظام بالكامل!

Exercise 3.36 (b). User Requirements vs. System Requirements

Discuss the difference between User Requirements and System Requirements.

Comparative Dimensional Matrix:

Dimension	User Requirements	System Requirements
Target Audience	Client executives, business managers, end-users, contract administrators.	Software architects, system designers, programmers, database engineers, testers.
Abstraction Level	High-level abstract statements of business services, operational benefits, and system goals.	Detailed, highly structured, and technical specifications of all software capabilities and constraints.

Dimension	User Requirements	System Requirements
Notation Used	Natural language prose, high-level workflow diagrams, and conceptual mock-ups.	Standardized technical language, structured templates, formal mathematical notations, DFDs, ERDs, and APIs.
Design Independence	Completely devoid of software design or architectural considerations.	Specifies precise external interfaces, data structures, and operational boundaries while avoiding internal algorithm bias.
Role in Lifecycle	Forms the primary basis for the commercial procurement contract and user acceptance criteria.	Forms the primary input for architectural design, module construction, and unit/integration verification.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1–§3.2.2, pp. 45–47; companion Slides 6–8 & Slide 140.

ملاحظة أكاديمية هامة (Academic Note)

التسلسل الهرمي: متطلبات المستخدم (*User Requirements*) تُكتب بلغة العميل ليفهم ويدفع، ومتطلبات النظام (*System Requirements*) تفصلها هندسياً ليفهم المبرمج ويبني.

Section 2: Curriculum Gap-Closing Synthesis (3 High-Value Items)

Synthesis 12. Use Case Stereotypes: <<include>> vs. <<extend>>

Compare and contrast the <<include>> and <<extend>> dependency stereotypes in Use Case modeling across execution condition, arrow direction, base use case completeness, and practical examples.

Comparative Breakdown:

Dimension	<<include>> Stereotype	<<extend>> Stereotype
Execution Condition	**Unconditional and Mandatory:** The included use case executes every single time the base use case runs.	**Conditional and Optional:** The extending use case executes only if a specific extension point condition evaluates to true.
Arrow Direction	Points **from the base use case TO the included use case** (Base ⟶ Included).	Points **from the extending use case TO the base use case** (Extension ⟶ Base).
Base Completeness	The base use case is incomplete without the included use case; it cannot finish its goal without it.	The base use case is fully complete on its own; it functions normally whether the extension triggers or not.
Primary Purpose	Code/behavior reuse; refactoring common sub-routines shared across multiple use cases into one place.	Encapsulating optional, exceptional, or variant behavior without modifying or cluttering the core base flow.
Concrete Example	Withdraw Cash has an <<include>> to Verify PIN (every withdrawal unconditionally requires PIN check).	Calculate Overdraft Charge has an <<extend>> to Withdraw Cash (triggers only if account balance is negative).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, pp. 58–59; UML 2.5 Specification.

ملاحظة أكاديمية هامة (Academic Note)

فخ السهم في الامتحان: أكبر خطأ شائع هو عكس اتجاه سهم <<extend>>! السهم في <<extend>> ينطلق دائماً من الحالة الإضافية متجهاً نحو الحالة الأصلية ليشير إلى نقطة التوسيع، بينما في <<include>> ينطلق من الأصلية نحو التابعة.

Synthesis 13. Requirements Verification vs. Requirements Validation

Compare Requirements Verification and Requirements Validation in software engineering, grounding the distinction in Barry Boehm's classic criteria.

Comparative Breakdown:

Dimension	Requirements Verification	Requirements Validation
Boehm's Criterion	*"Are we building the product right?"*	*"Are we building the right product?"*
Primary Focus	Internal consistency, structural conformance, mathematical completeness, and adherence to SRS standards.	External customer satisfaction, operational fidelity, and genuine alignment with real-world user intent.
Reference Standard	Evaluated against standard specifications, syntactic rules, and organizational templates (e.g., IEEE 830).	Evaluated against actual stakeholder operational needs, business processes, and organizational goals.
Primary Reviewers	Software quality engineers, system analysts, peer developers, and technical inspectors.	Clients, domain end-users, business managers, and external sponsors.
Techniques Used	Formal syntax checking, consistency matrix verification, DFD/ERD balancing, traceability inspections.	Prototyping demonstrations, user acceptance tests (UAT), operational walkthroughs, and field interviews.
Failure Consequence	The SRS is structurally defective, contradictory, untraceable, or unverifiable.	The software is built flawlessly according to the SRS, but solves the wrong problem and is rejected by the customer.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.7, pp. 92–94; B.B. Agarwal, S.P. Tayal & M. Gupta (2010), §4.1, p. 85 (Boehm's verification/validation questions).

ملاحظة أكاديمية هامة (Academic Note)

قاعدة بوهيم الخالدة:

- Verification ← هل كتبنا المواصفات بطريقة صحيحة هندسياً؟
- Validation ← هل كتبنا مواصفات النظام الصحيح الذي يريده المستخدم حقاً؟

Synthesis 14. Requirement Defect Classes: Inconsistent vs. Ambiguous vs. Incomplete

Analyze and contrast the three most dangerous classes of requirements defects: Inconsistent, Ambiguous, and Incomplete Requirements. Provide formal definitions, concrete examples, and primary detection methods.

Comparative Breakdown:

1. **Inconsistent Requirements (Contradictions):**
   - **Definition:** Two or more statements within the SRS assert mutually incompatible system states, functional behaviors, timing rules, or data representations.
   - **Illustrative example (created for this question):** One requirement says *"All user passwords shall expire after 30 days,"* while another says "*System administrator credentials shall never expire.*" These statements conflict if administrator credentials are included in "all user passwords."
   - **Detection Method:** Cross-referencing traceability matrices, automated decision tables, formal mathematical verification, and peer inspection sessions.
2. **Ambiguous Requirements (Multiple Interpretations):**
   - **Definition:** A requirement statement is phrased such that readers can interpret its meaning, operational bounds, or performance limits in multiple distinct ways.
   - **Concrete Example:** "*The database shall display transaction histories quickly and intuitively.*"
   - **Detection Method:** Independent party reviews (having two different developers explain how they would test the statement) and verifying whether quantifiable metrics exist.
3. **Incomplete Requirements (Omissions):**
   - **Definition:** The SRS fails to specify necessary system responses to specific input combinations, boundary conditions, hardware failures, or exceptional states.
   - **Concrete Example:** The SRS defines what happens when a student enters valid marks (1 to 100), but completely omits what the software should do when a user enters negative marks, letters, or leaves the field blank.
   - **Detection Method:** Boundary value analysis, state-transition matrix completeness checks, and structured review checklists targeting error handling and edge cases.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 73–75; IEEE Std 830-1998.

ملاحظة أكاديمية هامة (Academic Note)

خلاصة عيوب المتطلبات:

- Inconsistent: معلومتان متناقضتان تصطدمان ببعضهما.
- Ambiguous: معلومة واحدة تحتمل أكثر من تفسير.
- Incomplete: معلومة مفقودة تماماً سكت عنها المحلل.